

Foundations of Deep Learning

Lecture 06+07

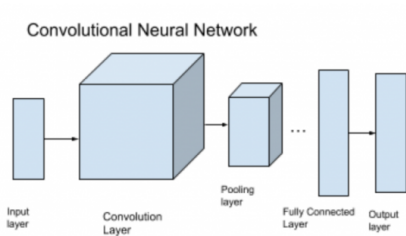
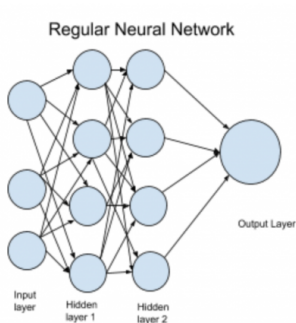
NEURAL NETWORK ARCHITECTURES

Aurelien Lucchi

Fall 2025

Constraining the Model

- ▶ Unlike MLPs, which are characterized by dense layers, convolutional NNs are made up of **sequential arrangement of convolutional and pooling layers**
- ▶ A single convolutional layer in turn comprises multiple filters/feature maps



Constraining the Model

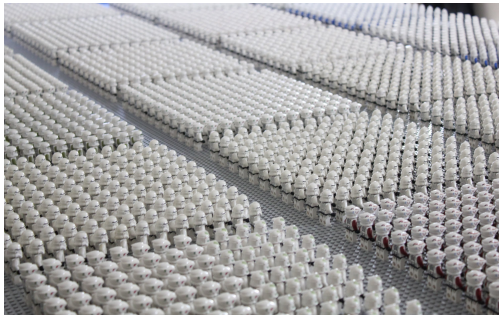
- ▶ No machine learning model can do well on all problems.
- ▶ Need to **constrain the function class** appropriately.
- ▶ Let's focus on the problem of image classification.



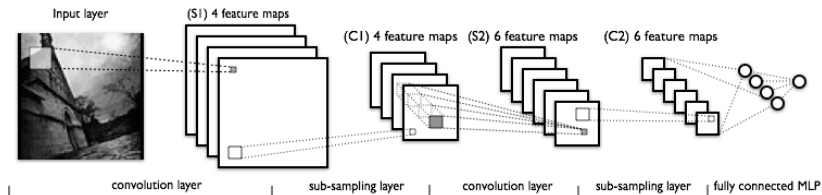
Can you name one property that is essential for this problem?

Example: Translation Invariance

- ▶ Translation invariance of images
 - ▶ image patches look the same, irrespective of their location
 - ▶ idea: extract translation invariant features



CNN: Buildings Blocks



► Four building blocks:

- Convolutional layer
- Pooling/subsampling layer
- Fully-connected layer
- Output: For binary classification, use a sigmoid function:

$$y_1 = P(Y = 1 | \mathbf{x}) = \frac{1}{1 + \exp[-\mathbf{w}^T \mathbf{x}]}$$

Section 1

CONVOLUTIONAL NETWORKS

Subsection 1

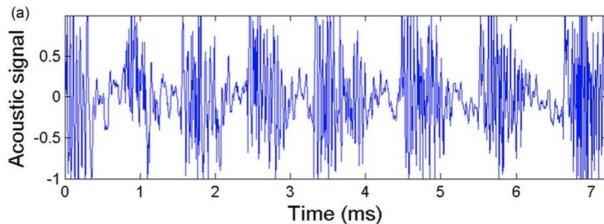
CONVOLUTION

Signals

We are interested in processing **signals**.

Signals may be sounds, images, measurement series, etc.

For concreteness: time signal $f : \mathbb{R} \rightarrow \mathbb{R}$.



Signal Transformation

Operating on a signal: **operators** or **transformations**

$$T : f \mapsto Tf, \quad Tf : \mathbb{R} \rightarrow \mathbb{R}$$

The result is a transformed signal.

Analogy: extract features from raw data representation.

Transformed signal extracts relevant information.

Integral Operators

How can we **represent** an interesting class of such operators?

Via Integrals!

Chose **kernel** $H : \mathbb{R}^2 \rightarrow \mathbb{R}$ and interval $-\infty \leq t_1 < t_2 \leq \infty$

This defines an integral operator

$$(Tf)(u) = \int_{t_1}^{t_2} H(u, t) f(t) dt$$

- ▶ f is the function being transformed
- ▶ $H(u, t)$ is the **kernel** of the operator, which determines the nature of the transformation, e.g., smoothing, scaling, or shifting f
- ▶ assuming that the integral exists (condition on f)

Fourier Transform

The Fourier transform $\mathcal{F}$ of a real-valued function f is an integral operator defined via

$$t_1 = -\infty, t_2 = \infty$$

$$H(u, t) = \exp(-2\pi i t u)$$

such that

$$(\mathcal{F}f)(u) := \int_{-\infty}^{\infty} e^{-2\pi i t u} f(t) dt$$

Convolution

Given two functions f, h , their convolution is defined as

$$\begin{aligned}(f * h)(u) &:= \int_{-\infty}^{\infty} h(u-t) f(t) dt \\&= \int_{-\infty}^{\infty} \underbrace{f(u-t)}_{\substack{\text{Move to desired time} \\ \& \text{ flip signal}}} h(t) dt \\&= (h * f)(u).\end{aligned}$$

where the second equality is due to a change of variables.

- ▶ corresponds to an integral operator with kernel $H(u, t) = h(u - t)$ operating on f
- ▶ by symmetry one can also think of $F(u, t) = f(u - t)$ operating on h

Convolution

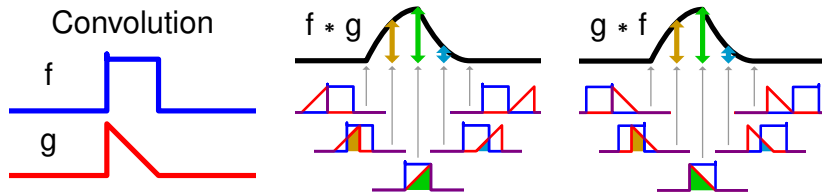


Figure: Source: wikipedia

Shift Equivariance

Shifted function f_Δ

$$f_\Delta(t) := f(t + \Delta)$$

Convolution is shift-equivariant, i.e.

$$\Rightarrow \boxed{f_\Delta * h = (f * h)_\Delta}$$

$$\begin{aligned}(f_\Delta * h)(u) &= \int_{-\infty}^{\infty} h(u - t) f(t + \Delta) dt \\ &\stackrel{t \mapsto t - \Delta}{=} \int_{-\infty}^{\infty} h(u + \Delta - t) f(t) dt \\ &= (f * h)(u + \Delta) = (f * h)_\Delta(u)\end{aligned}$$

Comments

1. The convolution operator is commutative, one can exchange the function and the kernels.
2. Its existence depends on integrability properties of f, h .
3. Typical use: $f = \text{signal}$, $h = \text{fast decaying kernel function}$, often of finite support.

Linear Shift-Equivariant Transforms

Instead of defining convolutions as a special case of an integral transform, one can take a principled approach and start with a question ...

How can we characterize **linear** shift-equivariant operators?

$$T(\alpha f + \beta g) = \alpha T f + \beta T g, \quad \forall f, g; \forall \alpha, \beta \in \mathbb{R}$$

$$(T f_{\Delta})(t) = (T f)(t + \Delta)$$

Linear Shift-Equivariant Transforms

Theorem 1

Any linear, translation-invariant transformation T can be written as a convolution with a suitable h .

Discrete Convolution

In practice: signals (digitally) sampled. Need to consider discrete case.

Let $f, h : \mathbb{Z} \rightarrow \mathbb{R}$. Define discrete convolution via

$$(f * h)[u] := \sum_{t=-\infty}^{\infty} f[t] h[u - t]$$

- ▶ use of rectangular brackets to suggest “arrays”
- ▶ typical choice of h : support over finite window, e.g. $h[t] = 0$ for $t \notin [t_{\min}; t_{\max}]$.

Gaussian Kernel

Let us define a small Gaussian kernel with support $[-2 : 2] \subset \mathbb{Z}$.

$$h[t] = \frac{1}{16} \begin{cases} 6 & t = 0 \\ 4 & |t| = 1 \\ 1 & |t| = 2 \\ 0 & \text{otherwise} \end{cases}$$

Because of finite support, convolution sum can be truncated

$$\begin{aligned} (f * h)[u] &= \sum_{t=u-2}^{u+2} f[t] h[u-t] = \sum_{t=-2}^2 h[t] f[u-t] \\ &= \frac{6f[u] + 4f[u-1] + 4f[u+1] + f[u-2] + f[u+2]}{16} \end{aligned}$$

Cross-correlation

Sibling to convolutions: **cross-correlation**.

In the discrete case:

$$(h \star f)[u] := \sum_{t=-\infty}^{\infty} h[t] f[u + t]$$

- ▶ sliding inner product
- ▶ difference to convolution: $u + t$ instead of $u - t$.
- ▶ flipped over kernel

$$\Rightarrow (h \star f) = (\bar{h} * f), \quad \text{where } \bar{h}[t] := h[-t]$$

Toeplitz Matrices

In practice: signal f and kernel h finitely supported

Without loss of generality assume $f[t] = 0$ for $t \notin [1 : n]$, $h[t] = 0$ for $t \notin [1 : m]$, $m \leq n$.

We can then think of f and h as vectors and define

$$(f * h) = \underbrace{\begin{pmatrix} h_1 & 0 & 0 & 0 & \dots & 0 & 0 \\ h_2 & h_1 & 0 & 0 & \dots & 0 & 0 \\ h_3 & h_2 & h_1 & 0 & \dots & 0 & 0 \\ \vdots & & & & & & \vdots \\ 0 & 0 & 0 & 0 & \dots & h_m & h_{m-1} \\ 0 & 0 & 0 & 0 & \dots & 0 & h_m \end{pmatrix}}_{:= \mathbf{H}_n^h \in \mathbb{R}^{(n+m-1) \times n}} \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ \dots \\ f_n \end{pmatrix}$$

Subsection 2

CONVOLUTION NETWORKS

Convolutional Networks: Goal

What is our goal with convolutional networks?

- ▶ a compositional (i.e. layered) architecture for signal processing
- ▶ exploiting **translation equivariance**
- ▶ exploiting locality and scale (e.g. temporal, spatial)
- ▶ learning (parameterized) kernel functions or **filters**
- ▶ increased efficiency relative to fully-connected DNNs

Convolutions in Higher Dimensions

Generalize concept of convolution to ...

- ▶ $2D$: e.g. images, spectrograms
- ▶ $3D$: e.g. color or multi-spectral images, voxel images, video
- ▶ or even higher dimensions

Replace vectors by

- ▶ matrices or fields (e.g. in discrete case)

$$(F * G)[i, j] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} F[i - k, j - l] \cdot G[k, l]$$

- ▶ tensors: for $3D$ and higher

Convolutions in Higher Dimensions

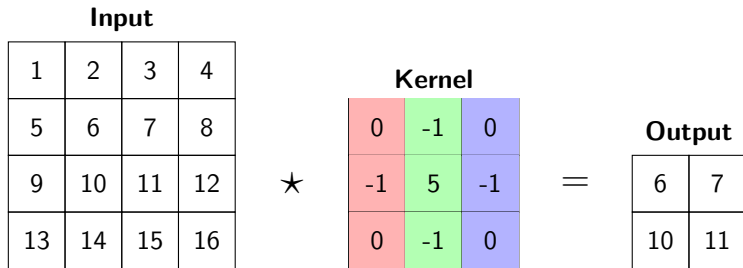
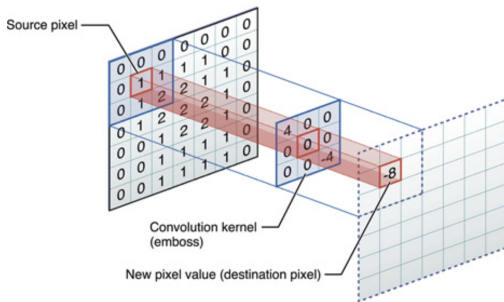


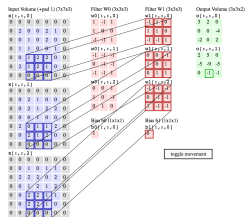
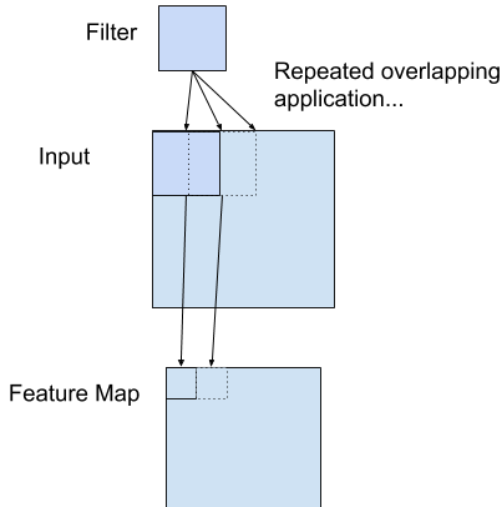
Figure: Illustration of a 2d convolution of a 4×4 input by a 3×3 kernel.

Convolutional Layers: Layout

- ▶ Convolved signal **inherits** topology of original signal.
- ▶ Hence: units in a convolutional layer are typically arranged on the same grid (1D, 2D, 3D, ...)



Convolutional Layers: Animation



[cs231n.github.io/
assets/conv-demo/
index.html](https://cs231n.github.io/assets/conv-demo/index.html)

Convolutional Layers: Border Handling

Different options for border handling

1. padding with zeros
2. only retain values from windows fully contained in support of signal f

Convolutional Layers

► **Convolution:**

- ▶ Mathematical operation on two functions (f and g)
- ▶ It produces a third function that is typically viewed as a modified version of one of the original function
- ▶ This operation can be used to detect edges in an image

-1	0	1
-2	0	2
-1	0	1

Horizontal

-1	-2	-1
0	0	0
-1	-2	-1

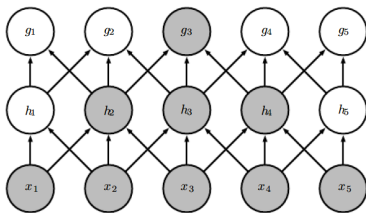
Vertical



Important: In a CNN, the filters are learned!

Nested convolutions & Receptive field

Schematic view of the notion of a **receptive field** in a deep network (input $x \rightarrow$ layer $h \rightarrow$ layer g)



- ▶ nesting of convolutions: receptive fields grow
- ▶ encourage network to first extract **localized** features
- ▶ subsequent layers: less and less localized features

Weight Sharing

- ▶ **MLP: No weight sharing**

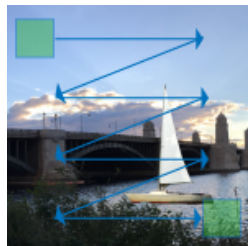
- ▶ each neuron is parametrized with weights $\mathbf{w} \in \mathbb{R}^n$

- ▶ **Weight Sharing**

- ▶ neurons share the same weights = compute same function
 - ▶ differ in location of their receptive field = **different input**
 - ▶ multiple channels = multiple filters

-1	0	1
-2	0	2
-1	0	1

Filter =
set of weights



Efficient Computations of Convolutional Activities

FFT (**Fast Fourier Transform**): compute convolutions fast(er).

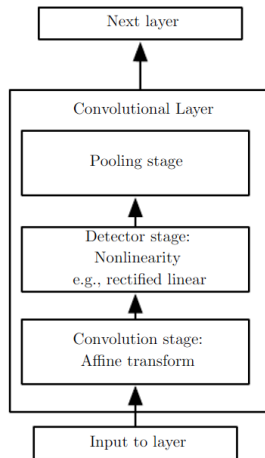
1. Fourier transform of signal $f \rightarrow (\mathcal{F}f)$ and kernel $h \rightarrow (\mathcal{F}h)$
2. pointwise multiplication and inverse Fourier transform

$$(f * h) = \mathcal{F}^{-1} ((\mathcal{F}f) \cdot (\mathcal{F}h))$$

3. FFT: signal of length n , can be done in $\mathcal{O}(n \log n)$
4. pays off, if many channels (amortizes computation of $\mathcal{F}f$)
5. small kernels ($m < \log n$): favor time/space domain

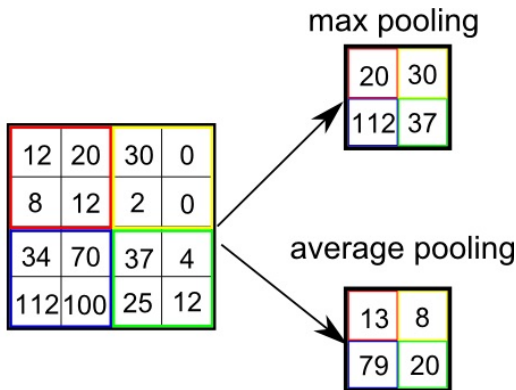
Convolutional Layers: Stages

1. **Non-linearities**: detector stage.
As always: scalar non-linearities
(activation function)
2. **Pooling** stage: locally combine activities



Pooling/subsampling layer

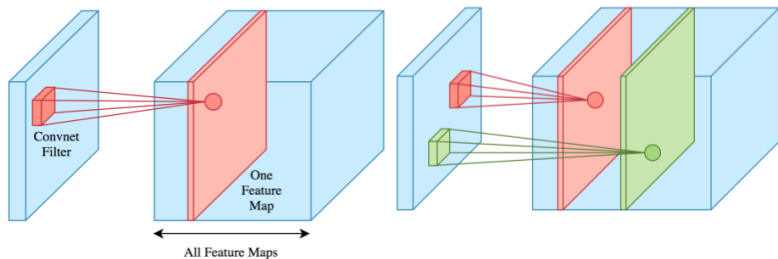
- ▶ Reduce size of convolutional layers by down-sampling
- ▶ Take average over window (e.g. 2x2)
- ▶ Common practice: **max pooling** = take maximum in window



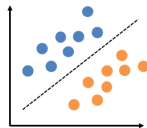
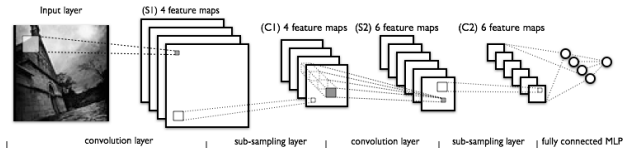
Channels

Learn multiple convolution kernels (or filters) = multiple **channels**

- ▶ typically: all channels use same window size
- ▶ channels form additional dimension for next layers (e.g. $2D \text{ signal} \times \text{channels} = 3D \text{ tensor}$)
- ▶ number of channels: design parameter



Fully-connected layer



- ▶ High-level reasoning
- ▶ Connects all neurons in the previous layer to **every** single neuron in the fully-connected layer
- ▶ Can be computed with a matrix multiplication

Pytorch Implementation

```
import torch.nn as nn
import torch.nn.functional as F

# Define a simple network with one convolutional layer
class SimpleConvNet(nn.Module):
    def __init__(self):
        super(SimpleConvNet, self).__init__()
        # One input channel (e.g. grayscale image),
        # one output channel, 3x3 kernel
        self.conv1 = nn.Conv2d(in_channels=1,
                                out_channels=1, kernel_size=3)

    def forward(self, x):
        # Apply convolution, then a nonlinearity (ReLU)
        x = self.conv1(x)
        x = F.relu(x)
        return x
```

Pytorch Implementation

```
# Create the network
net = SimpleConvNet()

# Create a random input tensor (batch_size=1, channels
    =1, height=5, width=5)
x = torch.randn(1, 1, 5, 5)

# Forward pass
output = net(x)

print("Input shape:", x.shape)
print("Output shape:", output.shape)
print("Output tensor:\n", output)
```

Subsection 3

CONVOLUTIONAL COMPUTER VISION

Visual Object Recognition

Important challenge for computer vision systems:

Visual **object recognition**: paradigmatic problem

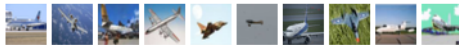
Convolutional networks have been used with remarkable success (**ImageNet** challenge)

- ▶ Pushing recognition to human-level accuracy
- ▶ Debatable of whether there is real understanding (adversarial examples, correlated features) of how the models work (lack of interpretability)

Visual Object Recognition: Examples

Examples from CIFAR dataset

airplane



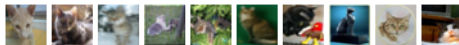
automobile



bird



cat



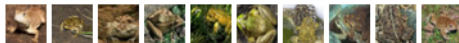
deer



dog



frog



horse



ship



truck



ImageNet

Data set: ImageNet (categories derived from WordNet)

... more than 20'000 categories

... more than 10 million hand-labeled images



Convolutions for Images

Color images

- ▶ pixel-based image representation: 3d.
- ▶ regular two dimensional, spatial sampling grid
- ▶ 3 color channels (color space such as RGB).

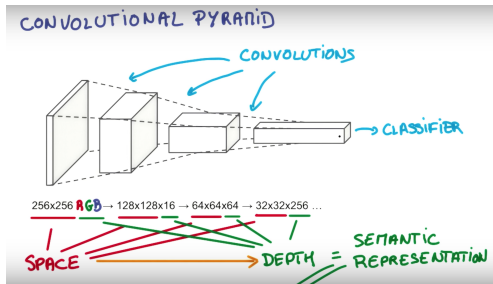
Convolutional kernels are usually operating on 3-*tensors*.

- ▶ channels: permutation-invariant, numbering arbitrary.
- ▶ typically fully connected across channels
- ▶ combined kernel over $r \times r$ window and m channels.

Pyramidal Architecture

Very common high-level model structure: **pyramid**

- ... stack convolutions, reducing spatial resolution of feature maps
- ... increasing the number of channels
- ... in the end: densely connected layer



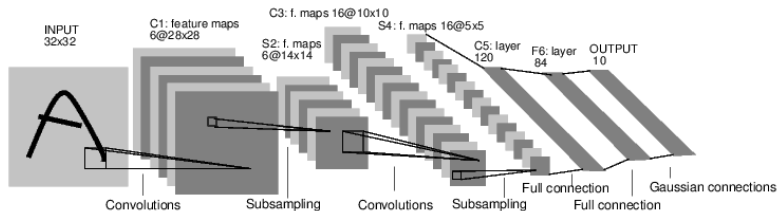
LeNet5

Classical model: LeNet5 (LeCun et al 1989, 1998)

C1/S2: 6 channels, 5×5 kernels, 2×2 subsampling (4704 units)

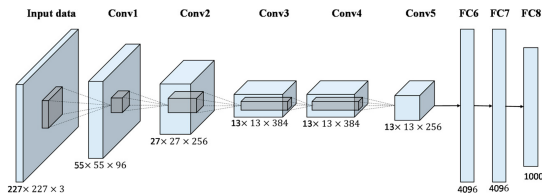
C3/S4: 16 channels, 6×6 kernels, 2×2 subsampling (1600 units)

C5: 120 channels; F6: fully-connected



AlexNet

More modern version: AlexNet (2012, 70k+ citations)



Parameter count:

384 to 384 channels with 3×3 window $\mapsto 1.3M+$ parameters

Layer FC6: $13 \times 13 \times 256$ tensor to 4096 $\mapsto 177M+$ parameters.

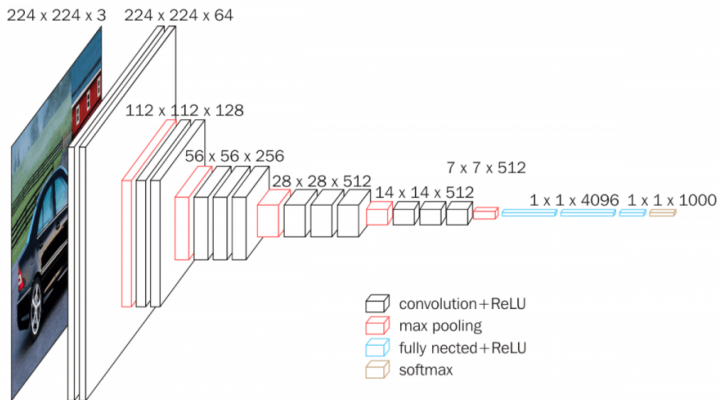
Architectural Considerations

So what matters with such vision architectures?

- ▶ control performance (many real-world applications require high throughput) $\implies$ avoid blow-up in model size
- ▶ make models **deep** (compositionality matters)
- ▶ make layers **wide** (allow for a sufficient number of channels)

Have to make trade-offs!

Very Deep Convolutional Networks: VGG



Traits of the VGG architecture

- ▶ very small convolution windows: 3×3
- ▶ avoids down-sampling and pooling
- ▶ significant increase in depth (relative to AlexNet, LeNet5)
- ▶ 3 layer blocks: $3 \cdot (3 \times 3) = 27 < 49 = (7 \times 7)$

Inception

Another popular model w/ deep stacking:

Inception Network (Google)

- (1) Uses many channels for high accuracy
- (2) Dimension reduction in between convolutions
- (3) $1 \times 1 \times m$ convolution $k \leq m$:

$$\mathbf{x}_{ij}^+ = \sigma(\mathbf{W}\mathbf{x}_{ij}), \quad \mathbf{W} \in \mathbb{R}^{k \times m}$$

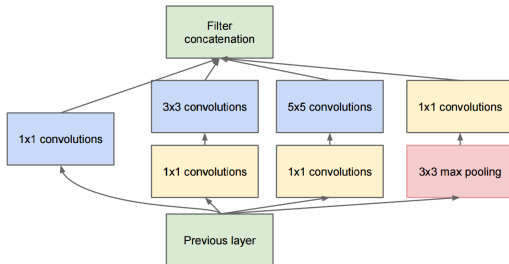
- ▶ 1×1 : i.e. no spatial convolution
- ▶ shared one-layer network transforms feature vectors across channels at each position
- ▶ instance of the network within a network design pattern

Inception: Block

Why chose, if one can have it all?

Multiple processing paths: trade-off of between channel dimensionality vs. window size solved as part of the learning

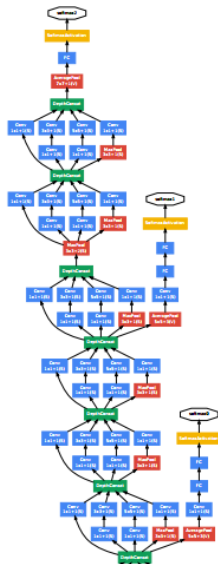
Create block structures, combined with dimension reduction.



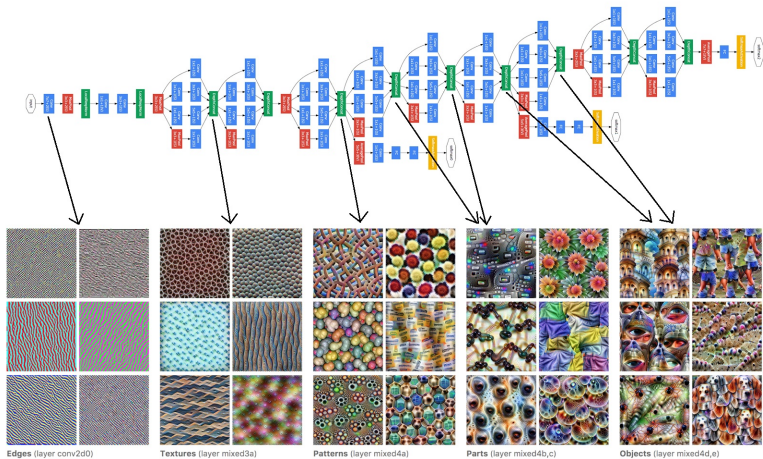
Inception: Model Architecture



Inception: Model Architecture



Inception: Interpretation



Subsection 4

CONVOLUTIONAL MODELS FOR NATURAL LANGUAGE

Embeddings

CNNs: successfully applied in natural language processing (NLP)

Typically: start with a sequence of **symbols** (characters, subword units, words) $\omega \in \Omega$

Map these symbols to vector space: **embedding**

$$\Omega \ni \omega \mapsto \mathbf{x}_\omega \in \mathbb{R}^n$$

$\implies$ nothing but a (possibly large) lookup table: entries are learnable parameters

word2vec

Word2vec: Embedding-based model of word representations

Input and output embedding:

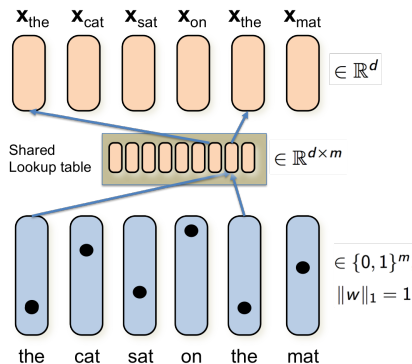
$$\Omega_{\text{in}} \ni \omega \mapsto \mathbf{x}_{\omega} \in \mathbb{R}^d \qquad \Omega_{\text{out}} \ni \nu \mapsto \mathbf{y}_{\nu} \in \mathbb{R}^d$$

Conditional **log-bilinear** model

$$\mathbb{P}(\nu \mid \omega) = \frac{\exp[\mathbf{x}_{\omega} \cdot \mathbf{y}_{\nu}]}{\sum_{\mu} \exp[\mathbf{x}_{\omega} \cdot \mathbf{y}_{\mu}]}$$

$\implies$ predicts word ν in the neighborhood of word ω .

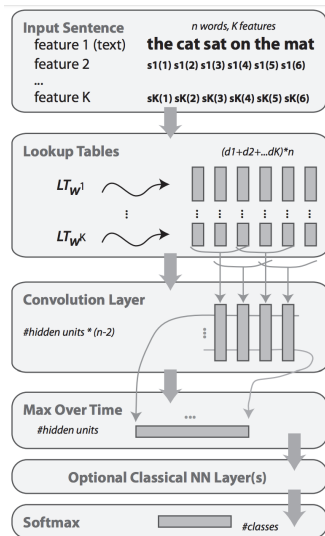
Variable Length Token Embeddings



$\implies$ conversion of one-hot vectors (symbol index) into embeddings via a shared lookup table

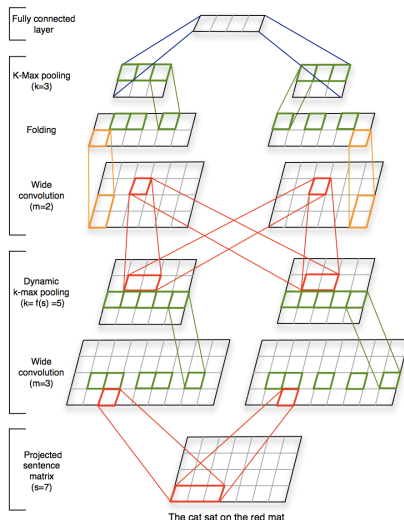
CNN Architectures for NLP

Classical model due to Collobert et al. 2008



CNN Architectures for NLP

Single-channel cross-pooling CNNs of Kalchbrenner et al. 2014



CNN Architectures: Discussion

Pros: Easy to design and diagnose. High degree of parallelism.

Cons: Limited dependency range. Fixed-length requires pooling.

Recently: Combinations of CNN with attention and memory units

...

Alternatives: recurrent neural networks, **transformers**

Section 2

CONNECTIVITY

Subsection 1

SHORT CONNECTIVITY

Residual Layers

Assume that we have a layer transfer function

$$g : \mathbb{R}^m \rightarrow \mathbb{R}^m.$$

We can parameterize it as

$$g(\mathbf{x}) = \mathbf{x} + f(\mathbf{x}; \boldsymbol{\theta}).$$

Identity function as the default; network can additively modify.

Initialize $f \approx 0$, Jacobian $\mathbf{J}_g = \mathbf{I} + \mathbf{J}_f \approx \mathbf{I}$.

Ideal for gradient propagation!

Linear Residuals

If the dimensions do not match, can use a linear network as default.

$$g(\mathbf{x}) = \mathbf{W}\mathbf{x} + f(\mathbf{x}; \boldsymbol{\theta}).$$

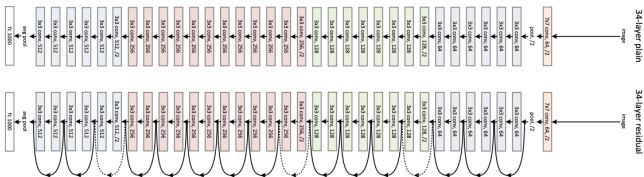
Initialize $f \approx 0$, Jacobian $\mathbf{J}_g = \mathbf{W} + \mathbf{J}_f \approx \mathbf{W}$.

Again: easy to initialize and well-behaved gradients.

Networks with Residual Layers

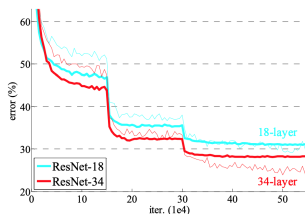
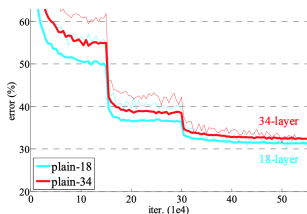
Use any standard DNN and add connectivity (branch and add)

Here: deep convolutional (VGG-style) network, augmented by residual connections (cf. arrows in diagram)



ResNets: Imagenet Results

Experimental comparison on ImageNet. Benefits of depth?



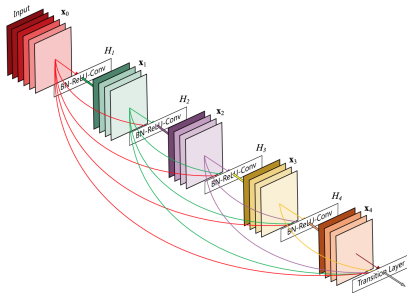
w/o residual connections: deeper networks is poorer

w/ residual connections: **advantage of depth**

Dense Connectivity

Dense blocks (Huan et al. 2017):

Connect layer output to *all* downstream layers



Concatenate activations, instead of adding (residual layers).

Section 3

NORMALIZATION

Normalization Techniques

Observation: poorly calibrated dynamic range of activations

⇒ poor backpropagation of errors (**vanishing gradients**)

How to measure dynamical range / variability?

1. Across many data points (e.g. mini batch)

⇒ **batch normalization**

2. Across units in the same layer

⇒ **layer normalization**

Batch Normalization: Motivation

Strong dependencies between weights in layers exist.

↪ Hard to find suitable learning rate.

- ▶ toy example: deep network with one unit per layer

$$y = w_1 \cdot \dots \cdot w_L x \implies y^{new} = \prod_{l=1}^L \left(w_l - \eta \frac{\partial \ell}{\partial w_l} \right) x$$

multiplying out leads to terms of up to order L .

- ▶ Gradients are products of $\mathcal{O}(L)$ terms:
 - ↪ If these gradients are small (less than 1), the product tends to zero as L increases, leading to vanishing gradients
 - ↪ If these gradients are large (greater than 1), the product grows exponentially, leading to exploding gradients.

Batch Normalization

[Ioffe & Szegedy, 2015]

Key idea: normalization is performed using statistics calculated from the **current mini-batch**

- ▶ fix layer l , fix set of examples $I \subseteq [1 : s]$
- ▶ mean activity, vector of standard deviation

$$\boldsymbol{\mu}^l := \frac{1}{|I|} \sum_{j \in I} \mathbf{z}^l[t], \quad \mathbf{z}^l[j] := (F^l \circ \dots \circ F^1)(\mathbf{x}[j])$$

$$\sigma_i^l := \sqrt{\delta + \frac{1}{|I|} \sum_{j \in I} (z_i^l[j] - \mu_i^l)^2}, \quad \delta > 0$$

- ▶ $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are functions of the weights: can be differentiated

Batch Normalization

[Ioffe & Szegedy, 2015]

- ▶ Normalized activities (cf. z -score in statistics)

$$\tilde{z}_i^l := \frac{z_i^l - \mu_i^l}{\sigma_i^l}, \quad \mathbb{E}[\tilde{z}_i^l] = \mathbf{0}, \quad \mathbb{E}[(\tilde{z}_i^l)^2] = 1$$

- ▶ Regain representational power

$$\hat{z}_i^l = \alpha_i^l \tilde{z}_i^l + \beta_i^l$$

- ▶ in principle: can exactly undo the batch normalization
- ▶ however: different learning dynamics, mean activation (and scale) can now be directly controlled

Layer Normalization

Same idea as batch normalization.

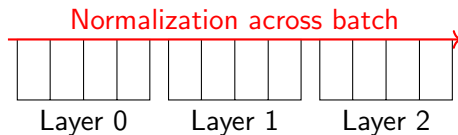
But: normalization is performed using statistics calculated for each input instance **across its features**.

$$\mu^l[t] := \frac{1}{m^l} \sum_{i=1}^{m^l} z_i^l[t]$$

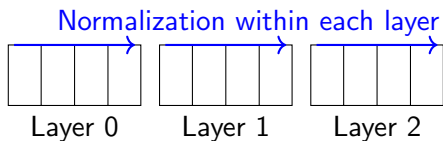
$$\sigma^l[t] := \sqrt{\delta + \frac{1}{m^l} \sum_{i=1}^{m^l} (z_i^l[t] - \mu^l[t])^2}, \quad \delta > 0$$

Batch vs. Layer Normalization: illustration

**Batch
Normalization**



**Layer
Normalization**



Batch vs. Layer Normalization

Batch normalization:

- (+) very effective, broadly used in vision
- (-) dependence on batch size, not suitable for all architectures

Layer normalization:

- (+) very effective, used in natural language (transformers, RNNs)
- (-) no consistent benefits

Section 4

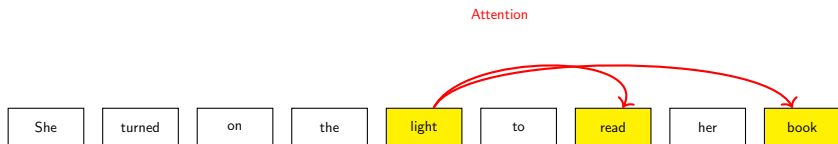
ATTENTION MODELS

Transformers in Deep Learning

- ▶ Transformers are a type of deep learning architecture introduced in the paper "Attention is All You Need" by Vaswani et al. (2017).
- ▶ They have become the **state-of-the-art** approach in many natural language processing (NLP) tasks, such as machine translation and language understanding.

Transformer Model: Attention Mechanism

- ▶ Unlike traditional recurrent neural networks (RNNs) or convolutional neural networks (CNNs), Transformers rely solely on **self-attention mechanisms** to process input data.
- ▶ Self-attention allows Transformers to capture **dependencies** between different words in a sentence.

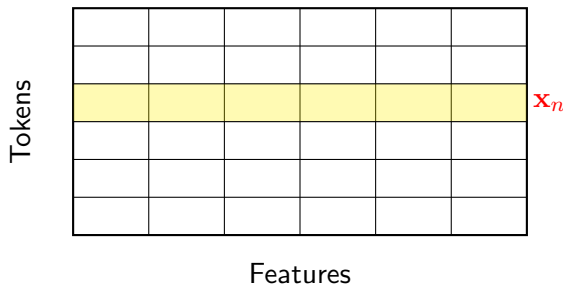


Transformer Model: Input data

Input data to a transformer is a set of vectors $\{\mathbf{x}_n\}$ of dimensionality d , where $n = 1, \dots, N$. We refer to these data vectors as **tokens**, where a token might, e.g., correspond to:

- ▶ A word within a sentence
- ▶ A patch within an image
- ▶ An amino acid within a protein

The elements x_{ni} of the tokens are called **features**.



Attention Mechanism

- ▶ **Goal:** map input tokens $\mathbf{x}_1, \dots, \mathbf{x}_N$ to another set $\mathbf{y}_1, \dots, \mathbf{y}_N$ that captures a richer semantic structure.
- ▶ With attention, this dependence should be stronger for those inputs $\mathbf{x}_m$ that are particularly important for determining the modified representation of $\mathbf{y}_n$.

Mathematically:

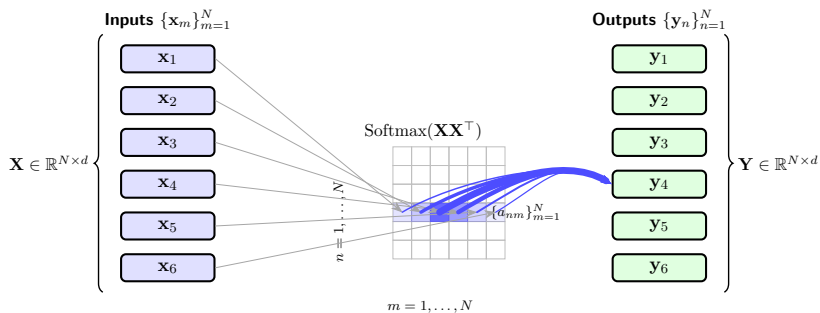
$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m, \quad a_{nm} = \frac{\exp(\mathbf{x}_n^\top \mathbf{x}_m)}{\sum_{k=1}^N \exp(\mathbf{x}_n^\top \mathbf{x}_k)}$$

where a_{nm} are the attention coefficients.

$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}$

$\text{Softmax}(\mathbf{A})$ applies the exponential function to each element of a matrix $\mathbf{A}$ and then normalizes each row so that the sum of the elements in each row equals one.

Attention Mechanism



Attention Mechanism: Adjustable Parameters

We redefine the modified feature vectors as the result of applying a linear transformation to the original vectors, expressed as $\mathbf{X}\mathbf{U}$, where $\mathbf{U}$ is a $d \times d$ matrix containing learnable weight parameters.

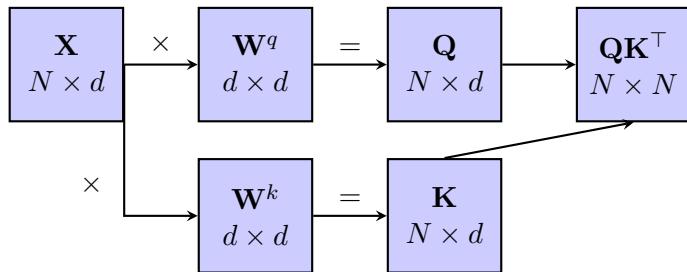
$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{X}\mathbf{U}\mathbf{U}^\top\mathbf{X}^\top)\mathbf{X}\mathbf{U}$$

Add more flexibility by defining 3 matrices (query, key, value):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^q \quad \mathbf{K} = \mathbf{X}\mathbf{W}^k \quad \mathbf{V} = \mathbf{X}\mathbf{W}^v. \quad (1)$$

$$\rightsquigarrow \text{Matrix form: } \mathbf{Y} = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$$

Attention Mechanism: Adjustable Parameters



- ▶ Matrices $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_v \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d_v \times d_v}$ are learnable parameters, and each layer is initialized with an independent set of weights
- ▶ Figure assumes $d_v = d_k = d$ for simplicity

Attention Mechanism vs CNN

$$\mathbf{Y} = \text{Softmax} \left\{ \mathbf{QK}^T \right\} \times \mathbf{V}$$

$N \times d$ $N \times N$ $N \times d$

- ▶ In contrast to a conventional neural network, where activations are multiplied by fixed weights, the signal paths in this context involve multiplicative interactions between activation values.
- ▶ Instead of static weights, the activations here are dynamically multiplied by data-dependent attention coefficients.

Attention Mechanism: Multihead attention

Head: Attention layer described so far enables the output vectors to focus on input-specific patterns and is referred to as an **attention head**

↪ there might be multiple patterns of attention that are relevant at the same time.

Concatenation: Concatenate outputs from multiple attention heads and apply a linear transformation to generate the final output.

Let head_i denote the output of the i -th head. Then, the multi-head attention output is:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \mathbf{W}^O$$

Here, $\mathbf{W}^O \in \mathbb{R}^{hd \times d_{\text{model}}}$ is the output projection matrix.

Attention Mechanism

```
1 def scaled_dot_product_attention(Q, K, V):
2     """
3     Q, K, V: (batch_size, seq_len, embed_dim)
4     Returns:
5         output: (batch_size, seq_len, embed_dim)
6         attn_weights: (batch_size, seq_len, seq_len)
7     """
8     d_k = Q.size(-1) # embed_dim in single-head scenario
9
10    scores = torch.matmul(Q, K.transpose(-2, -1))/(d_k**0.5)
11    attn_weights = torch.softmax(scores, dim=-1)
12
13    output = torch.matmul(attn_weights, V)
14
15    return output, attn_weights
```


Attention Mechanism

```
1 class SingleHeadSelfAttention(nn.Module):
2     def __init__(self, embed_dim):
3         super().__init__()
4         self.embed_dim = embed_dim
5
6         # Linear projections for Q, K, V (all output embed_dim)
7         self.W_Q = nn.Linear(embed_dim, embed_dim)
8         self.W_K = nn.Linear(embed_dim, embed_dim)
9         self.W_V = nn.Linear(embed_dim, embed_dim)
10
11         # Final linear projection
12         self.W_O = nn.Linear(embed_dim, embed_dim)
13
14     def forward(self, x):
15         """x: (batch_size, seq_len, embed_dim)
16         Returns:
17             out: (batch_size, seq_len, embed_dim)
18             attn_weights: (batch_size, seq_len, seq_len)
19         """
20         # Project inputs x
21         Q = self.W_Q(x)
22         K = self.W_K(x)
23         V = self.W_V(x)
24
25         # Scaled dot-product attention (single head)
26         attn_output, attn_weights = scaled_dot_product_attention(Q, K, V)
27
28         # Final projection
29         out = self.W_O(attn_output) # (batch_size, seq_len, embed_dim)
30
31         return out, attn_weights
```

Transformer Model

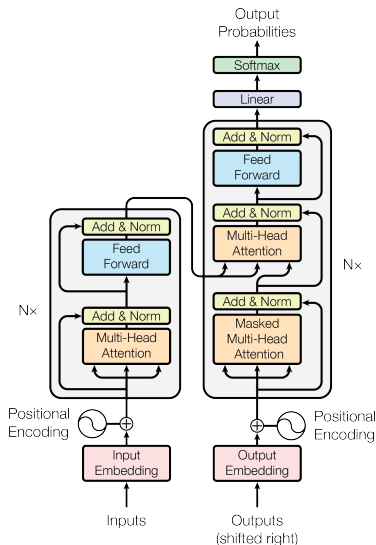


Figure: A single Transformer block. Source: Attention Is All You Need, Vaswani et al.

Transformer Model (simplified)

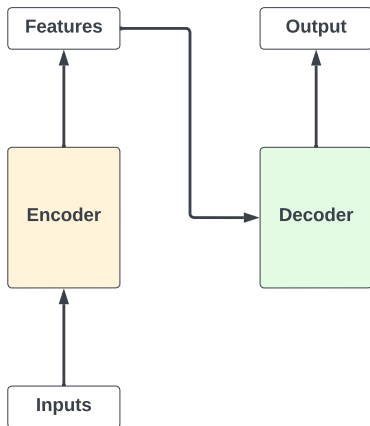
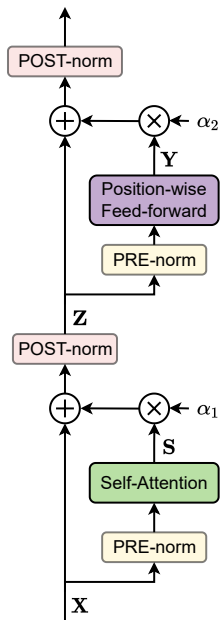


Figure: A single Transformer block: The encoder extracts features from an input sentence, and the decoder uses the features to produce an output sentence (e.g. for a translation task). The encoder is not necessary for all tasks.

Block of an Encoder



Self-attention

Given an input sequence $\mathbf{X} \in \mathbb{R}^{n \times d_v}$, with n tokens of dimension d_v , the (single-head) self-attention is defined as:

$$\mathbf{S}^\ell := \mathbf{A}^\ell \mathbf{X}^\ell \mathbf{W}^V, \text{ where } \mathbf{A}^\ell = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{X}^\ell \mathbf{W}^Q \left(\mathbf{X}^\ell \mathbf{W}^K \right)^\top \right),$$

where the softmax function is applied independently across each row, and the superscript ℓ indexes the ℓ -th layer.

- Matrices $\mathbf{X}^\ell \mathbf{W}^Q$, $\mathbf{X}^\ell \mathbf{W}^K$, $\mathbf{X}^\ell \mathbf{W}^V$ are referred to as queries, keys and values, respectively.

Note on nomenclature

The names queries, keys and values come from the field of information retrieval systems:

- ▶ The query is the question or request you're asking.
- ▶ The keys are indexed labels or features that allow the system to understand where the relevant information is.
- ▶ The values are the actual content or information retrieved once a match is made.

Block of a Encoder (2)

The complete encoder block can be written recursively as:

$$\begin{aligned}\mathbf{Z}^\ell &= \alpha_1 \mathbf{S}^\ell + \mathbf{X}^\ell \\ \mathbf{Y}^\ell &= \sigma(\mathbf{Z}^\ell \mathbf{W}^{F_1}) \mathbf{W}^{F_2} \\ \mathbf{X}^{\ell+1} &= \alpha_2 \mathbf{Y}^\ell + \mathbf{Z}^\ell,\end{aligned}$$

where the introduced α_1, α_2 parameters indicate the strength of the residual block, $\mathbf{W}^{F_1}, \mathbf{W}^{F_2} \in \mathbb{R}^{d_v \times d_v}$ are matrices of learnable parameters; we set $\mathbf{X}^0 := \mathbf{X}$, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is an activation function.

Block of a Decoder

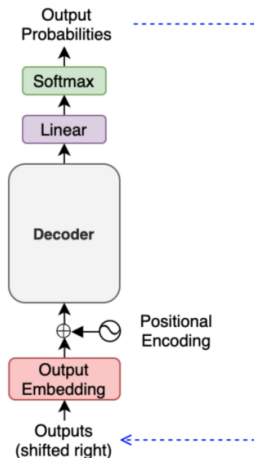


Figure: The decoder generates tokens sequentially, where each token it produces becomes the input for generating the next token. Put differently, the previous decoder output serves as the final segment of the subsequent decoder input. Source: <https://kikaben.com/transformers-encoder-decoder/>

Sampling Strategies

Problem Setup:

- ▶ A decoder Transformer outputs a probability distribution over the next token $\mathbf{y}_n$, given previous tokens $\mathbf{y}_{1:n-1}$.
- ▶ To find the most probable sequence of length N , we would ideally maximize:

$$\prod_{n=1}^N p(\mathbf{y}_n \mid \mathbf{y}_{1:n-1}),$$

$\rightsquigarrow \mathcal{O}(K^N)$ total sequences if K is the vocabulary size.

Sampling Strategies

Greedy Search:

- ▶ Select the highest-probability token at each step.
- ▶ Deterministic; does **not** necessarily yield the globally most probable sequence.

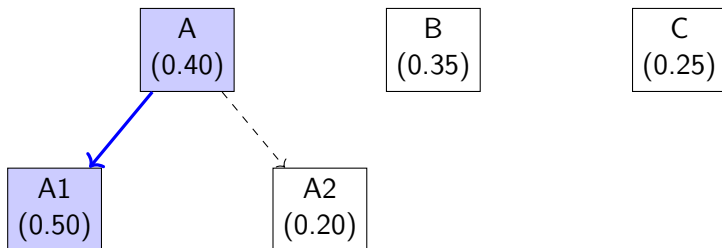
Beam Search:

- ▶ Maintain multiple hypotheses (sequences) simultaneously.
- ▶ At each step, expand each hypothesis with all possible next tokens.
- ▶ Keep the top B expansions by probability for the next step.
- ▶ Balances coverage of possible sequences with reduced computation compared to exhaustive search.

Greedy Search (GS) Example

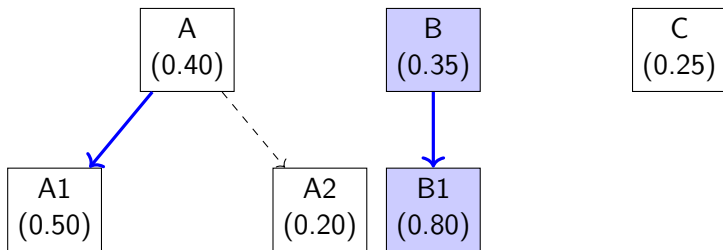
Given three tokens and their probabilities:
A (0.4), B (0.35), C (0.25).

- ▶ **Step 1:** GS selects the highest probability token (A).
- ▶ **Step 2 (after A):** Extension probabilities:
 - ▶ A1: 0.50
 - ▶ A2: 0.20
- ▶ **Final Sequence:** A, A1 with overall probability $0.40 \times 0.50 = 0.20$.



Beam Search Example (Beam Width = 2)

- ▶ **Step 1:** Beam search keeps the top 2 tokens: A and B.
- ▶ **Step 2:**
 - ▶ From A:
 - ▶ A1: 0.50 → Sequence: A, A1 ($0.40 \times 0.50 = 0.20$)
 - ▶ A2: 0.20
 - ▶ From B:
 - ▶ B1: 0.80 → Sequence: B, B1 ($0.35 \times 0.80 = 0.28$)
- ▶ **Final Selection:** Beam search chooses the sequence with the highest overall probability, which is **B, B1**.



Beam Search Example (Beam Width = 2)

```
1 def beam_search(initial_probs, expansions, beam_width, steps):
2     """
3     initial_probs: dict[token, float] of starting token probabilities
4     expansions: dict[token, dict[next_token, float]] of conditional
5                 probabilities
6     beam_width: int, how many hypotheses to keep at each step
7     steps: int, total number of time steps (including the initial one)
8     """
9     # Initialize beam with the first-step tokens
10    beam = [(token, prob) for token, prob in initial_probs.items()]
11    # Keep only the top beam_width
12    beam = sorted(beam, key=lambda x: x[1], reverse=True)[:beam_width]
13
14    # Expand for the remaining steps
15    for _ in range(1, steps):
16        all_candidates = []
17        for seq, seq_prob in beam:
18            last = seq[-1]
19            # For each possible next token from 'last'
20            for next_token, next_prob in expansions.get(last, {}).items():
21                new_seq = seq + [next_token]
22                new_prob = seq_prob * next_prob
23                all_candidates.append((new_seq, new_prob))
24            # Select the top beam_width sequences
25            beam = sorted(all_candidates, key=lambda x: x[1], reverse=True)[:
26                           beam_width]
27
28    return beam
```

Beam Search Example (Beam Width = 2)

```
1  if __name__ == "__main__":
2      # --- Example setup ---
3      # Step 1 probabilities
4      initial = {
5          "A": 0.40,
6          "B": 0.35,
7          "C": 0.25
8      }
9      # Step 2 expansions
10     expansions = {
11         "A": {"A1": 0.50, "A2": 0.20},
12         "B": {"B1": 0.80},
13         # "C" has no expansions in this example
14     }
15
16     beam_width = 2
17     total_steps = 2
18
19     final_beam = beam_search(initial, expansions, beam_width, total_steps)
20
21     print("Final beam (top 2 sequences):")
22     for seq, prob in final_beam:
23         print(f"  Sequence: {seq}, Probability: {prob:.2f}")
24
25     # And the best sequence is:
26     best_seq, best_prob = final_beam[0]
27     print(f"\nBest sequence: {best_seq} with probability {best_prob:.2f}")
```

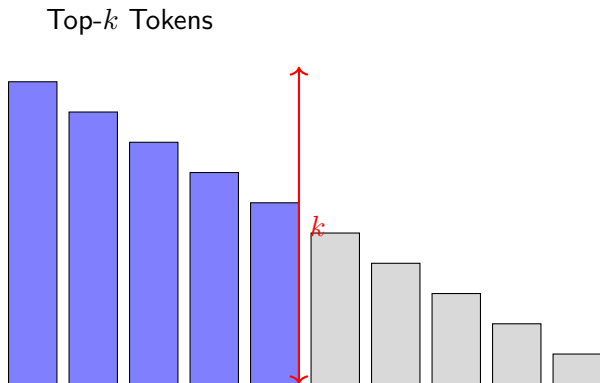
Sampling Strategies

Sampling

- ▶ Rather than searching for the single most likely sequence, we can generate each token in turn by sampling from the model's softmax distribution.
- ▶ Two variants: Top- k Sampling, Top- p Sampling

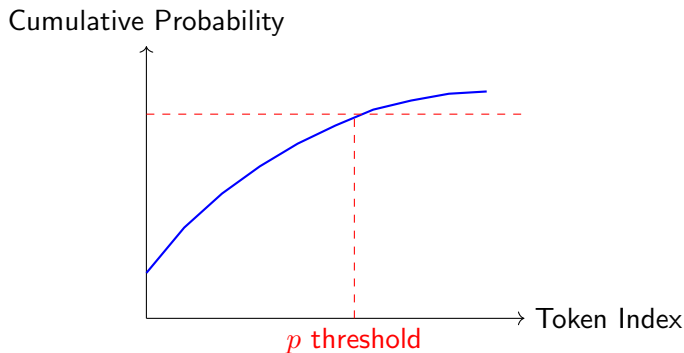
Top-k Sampling

- ▶ At each generation step, only the top k tokens with the highest probabilities are considered.
- ▶ The rest of the probability mass is discarded.
- ▶ This fixed-size truncation can reduce randomness but may limit diversity.



Top- p (Nucleus) Sampling

- ▶ Instead of a fixed number of tokens, top- p sampling selects the smallest set of tokens whose cumulative probability exceeds a threshold p .
- ▶ Number of tokens considered varies dynamically.



Comparison

- ▶ **Top-k Sampling:**

- ▶ Uses a fixed number of tokens.
- ▶ Simpler but may discard tokens that, while lower in probability, could be relevant.

- ▶ **Top-p Sampling:**

- ▶ Adapts to the probability distribution by selecting a dynamic set of tokens.
 - ▶ Can produce more varied and context-sensitive results.
- ▶ Both methods help balance the trade-off between diversity and coherence in text generation.

Applications of Transformer Architectures

- ▶ Transformers have found success in a wide range of applications beyond NLP, including:
 - ▶ Machine Translation
 - ▶ Image Generation (e.g., DALL-E)
 - ▶ Speech Recognition
 - ▶ Protein Folding
- ▶ They excel in tasks requiring sequence-to-sequence modeling.

Applications of Transformer Architectures - NLP

Tokenization is the process of converting raw text into a sequence of discrete units (tokens) that can be processed by a language model.

Key Ideas:

- ▶ Tokens can be characters, subwords, or full words.
- ▶ Subword tokenization offers a balance between flexibility and efficiency.
- ▶ Reduces vocabulary size while preserving semantic granularity.

Applications of Transformer Architectures - NLP

Tokenization example using GPT2

Token	ID	L2 Norm	Embedding Vector
Like	7594	3.30	$[-0.166, -0.080, 0.086, \dots]$
a	257	2.57	$[-0.051, 0.006, 0.047, \dots]$
fish	5916	3.26	$[0.027, 0.100, 0.066, \dots]$
out	503	2.71	$[-0.048, -0.044, 0.034, \dots]$
of	286	2.59	$[-0.057, 0.018, 0.033, \dots]$
water	1660	3.05	$[0.065, 0.020, 0.023, \dots]$

Applications of Transformer Architectures - NLP

Masking: Prevent tokens from attending to future positions during training.

Unmasked Attention:

$$\mathbf{A} = \frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^\top, \quad \mathbf{Q} = \mathbf{X} \mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X} \mathbf{W}^K$$

Causal Mask:

$$\mathbf{M}_{ij} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Masked Attention:

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{M}, \quad \mathbf{W} = \text{softmax}(\tilde{\mathbf{A}})$$

Effect: Enforces autoregressive generation by hiding future tokens.

Applications of Transformer Architectures - NLP

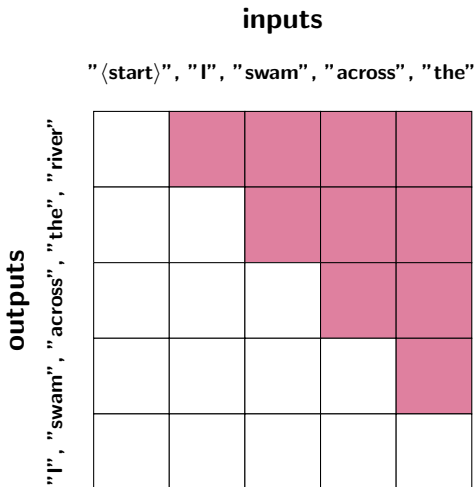


Figure: Illustration of the mask matrix used in masked self-attention, where attention weights for the purple entries are set to zero. Note the addition of a start token at the beginning of the input to ensure a consistent alignment between input and output tokens.

Applications of Transformer Architectures - Vision

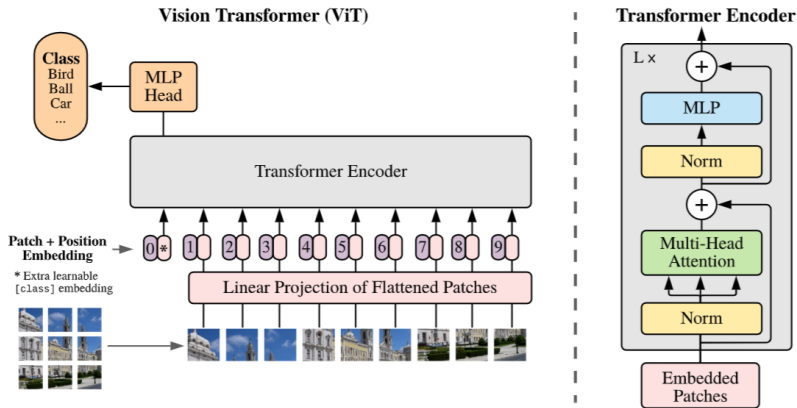


Figure: Vision Transformers (Dosovitskiy et al. 2021)

Shortcomings of Transformer Architectures

When it comes to **reasoning tasks**, such as logical reasoning, mathematical problem solving, commonsense reasoning, and structured thinking, Transformers still face some significant challenges, such as:

- ▶ Poor handling of multi-step reasoning
- ▶ Difficulty with symbolic and mathematical reasoning
- ▶ Lack of explicit reasoning mechanisms
- ▶ Challenges in commonsense and world knowledge reasoning
- ▶ Difficulty with logical consistency
- ▶ Inability to perform causal and counterfactual reasoning

Chain-of-thought Wei et al. (2022)

Introduced in Wei et al. (2022), chain-of-thought (CoT) prompting enables complex reasoning capabilities by generating intermediate reasoning steps.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

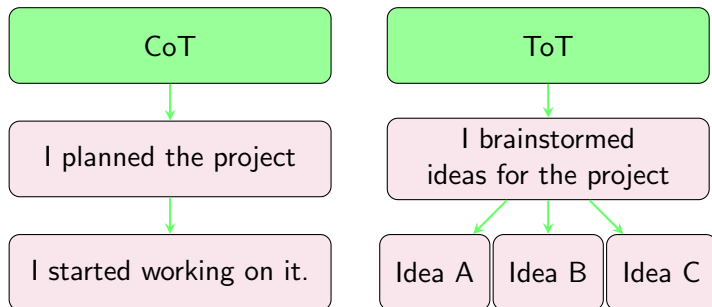
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Tree-of-thought Yao et al. (2023)



Tree-of-thought Yao et al. (2023)

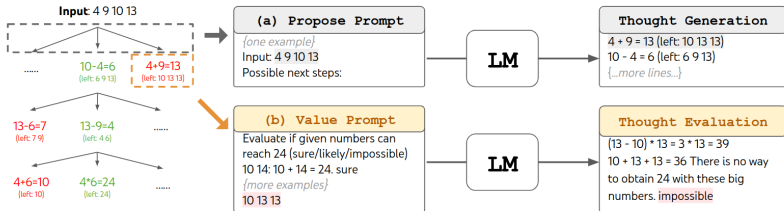


Figure: ToT in a game of 24. The LM is prompted for (a) thought generation and (b) valuation.

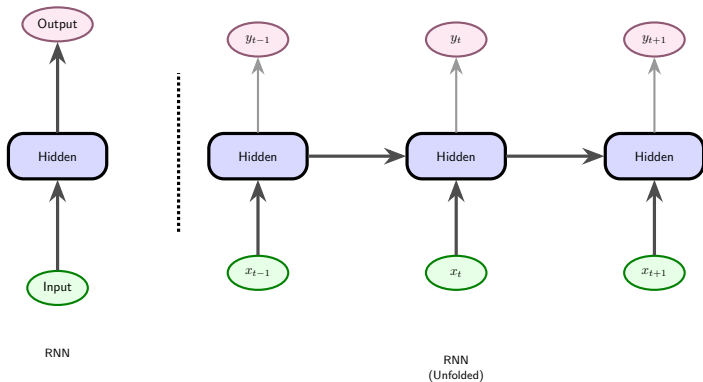
Try it yourself: <https://www.4nums.com/>.

Section 5

RNNs AND SSMs

Recurrent Neural Network (RNN)

Recurrent Neural Networks process sequential data by maintaining a hidden state that captures information from previous time steps to influence future outputs.



Recurrent Neural Network (RNN)

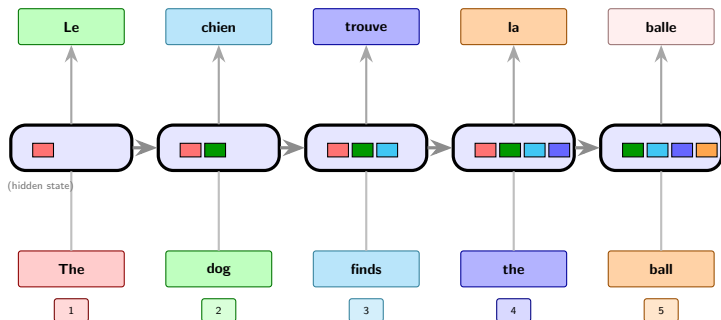


Figure: Illustration of sequence-to-sequence translation using a state space model. Each block represents a hidden state that evolves across time steps, mapping English tokens (bottom) to French outputs (top). The colored bars indicate how internal state representations accumulate information over time.

State Space Model (SSM)

- ▶ A SSM provides a compact way to describe the dynamics of a system using a set of first-order differential equations.
- ▶ It represents how an internal (hidden) state evolves over time in response to an input signal, and how this state produces an observable output.

State Space Model (SSM)

Continuous-Time Formulation At any time t , we define:

- ▶ $\mathbf{h}(t) \in \mathbb{R}^n$: the **state vector**, representing the internal memory or latent dynamics of the system,
- ▶ $\mathbf{x}(t) \in \mathbb{R}^p$: the **input vector**,
- ▶ $\mathbf{y}(t) \in \mathbb{R}^m$: the **output vector**.

The evolution of the system is governed by two coupled equations:

$$\text{State equation: } \mathbf{h}'(t) = \mathbf{A} \mathbf{h}(t) + \mathbf{B} \mathbf{x}(t),$$

$$\text{Output equation: } \mathbf{y}(t) = \mathbf{C} \mathbf{h}(t) + \mathbf{D} \mathbf{x}(t).$$

State Space Model (SSM)

Interpretation

- ▶ The **state equation** models how the hidden state $\mathbf{h}(t)$ changes over time.
- ▶ The **output equation** defines how the observable output $\mathbf{y}(t)$ is generated from the hidden state and current input.

Matrices

- ▶ **A**: the **state transition matrix**, determining how the current state influences its rate of change;
- ▶ **B**: the **input matrix**, describing how inputs affect the state dynamics;
- ▶ **C**: the **output matrix**, mapping the hidden state to the observed output;
- ▶ **D**: the **feedthrough matrix**, capturing any direct influence of the input on the output.

State Space Model (SSM)

Mamba [Gu and Dao(2024)] is a selective State Space Model (SSM) that makes the state dynamics input-dependent, enabling adaptive sequence modeling with linear-time complexity.

Given a fixed sampling interval Δ , we discretize the system via a zero-order-hold (ZOH) rule (for instance) to obtain:

$$\mathbf{A}_d = \exp(\Delta \mathbf{A}), \quad \mathbf{B}_d = (\Delta \mathbf{A})^{-1} (\exp(\Delta \mathbf{A}) - I) \mathbf{B}.$$

The resulting discrete-time recurrence is:

$$\mathbf{h}_{t+1} = \mathbf{A}_d \mathbf{h}_t + \mathbf{B}_d \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C} \mathbf{h}_t.$$

Acknowledgment

The part of this lecture about Transformers was made following the excellent book "Deep Learning, Foundations and Concepts" by Christopher M. Bishop and Hugh Bishop.

References



Albert Gu and Tri Dao.

Mamba: Linear-time sequence modeling with selective state spaces.

In **First Conference on Language Modeling**, 2024.